Operating System Lab Assignment 1

Assignment 1(a):-

Write a C program to create a child process using the system call *fork( )*. From the child process, display the PID and PPID and then call again the *fork( )* to create a grandchild and engage him to display your roll no. From parent display the PID and PPID of all the processes and display the count of total no. of child processes created also the number of grandchild processes created. Demonstrate the use of *exit(0)* and *exit(1)*.

Ans:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid1, pid2;
    int child_count = 0;
    int grandchild_count = 0;

    pid1 = fork();

    if (pid1 < 0) {
        exit(1);
    } else if (pid1 == 0) {
        printf("Child Process - PID: %d, PPID: %d\n", getpid(), getppid());

        pid2 = fork();
        if (pid2 < 0) {
            exit(1);
        } else if (pid2 == 0) {
            printf("Grandchild Process - Roll No: CSB23017\n");
            exit(0);
        } else {
            exit(0);
        }
    } else {
        child_count = 1;
        grandchild_count = 1;
        printf("Parent Process - PID: %d, PPID: %d\n", getpid(), getppid());
        printf("Total child processes: %d\n", child_count);
        printf("Total grandchild processes: %d\n", grandchild_count);
        exit(0);
    }
    return 0;
}
```

```
Parent Process - PID: 10, PPID: 7
Total child processes: 1
Total grandchild processes: 1
Child Process - PID: 18, PPID: 10
Grandchild Process - Roll No: CSB23017
```

Assignment 1(b):-

Write a C program like the assignment 1(a). But here use the system call *wait()* system to synchronize the execution of parent program in your program until child process finishes. Now write a function to print to find out who is logging in the local machine that is partially equivalent to the cmd "w" or "who" and engage the grandchild to print the output into the stdout.

Ans:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <utmp.h>

void print_users() {
    struct utmp *entry;
    setutent();
    while ((entry = getutent()) != NULL) {
        if (entry->ut_type == USER_PROCESS) {
            printf("Logged in user: %s\n", entry->ut_user);
        }
    }
    endutent();
}

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        pid_t gc_pid = fork();
        if (gc_pid == 0) {
            print_users();
            exit(0);
        }
        wait(NULL);
        exit(0);
    } else {
        wait(NULL);
        printf("Parent: Child process finished.\n");
    }
    return 0;
}
```

```
Parent: Child process finished.
```

Assignment 1(c):-

Write a C program like the assignment 1(b) and overlay a user designed program into the address space of the child process using *execv()* system call. Again use *wait()* system call to synchronize the execution of parent process in your program until child process finishes. Here use the macro WIFEXITED to capture the returned status of the child in parent process. Also demonstrate the use of argument vector to print the program name by the child process.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    pid_t pid = fork();
    int status;

    if (pid < 0) {
        exit(1);
    } else if (pid == 0) {
        printf("Child: Executing program %s\n", argv[0]);
        char *args[] = {"/bin/ls", "-l", NULL};
        execv(args[0], args);
        exit(1);
    } else {
        wait(&status);
        if (WIFEXITED(status)) {
            printf("Parent: Child exited with status %d\n", WEXITSTATUS(status));
        }
    }
    return 0;
}
```

```
Child: Executing program ./jdoodle
total 36
-rwxrwxrwx 1 root root 15784 Dec 20 13:30 jdoodle
-rw-r--r-- 1 root root   630 Dec 20 13:27 main.c
drwxr-xr-x 2 root root  4096 Dec 20 13:30 pacman
-rw-r--r-- 1 root root   986 Dec 20 13:29 question_1a.c
-rw-r--r-- 1 root root   756 Dec 20 13:29 question_1b.c
-rw-r--r-- 1 root root   556 Dec 20 13:29 question_1c.c
Parent: Child exited with status 0
```